

B 3.1.4 Classes and instantiation



B3.1.4 Construct code to define classes and instantiate objects.

This section covers defining classes, creating objects, and the role of constructors. Constructors initialize an object's state by setting initial attribute values.

Learning Objectives Java

By the end of this lesson, students will be able to:

- Define the syntax for declaring a class in Java.
- Identify and declare instance variables (attributes) within a Java class.
- Define a constructor within a Java class and explain its purpose in initialising object state.
- Write code to instantiate objects (create instances) of a defined Java class using the `new` keyword.
- Explain the role of the constructor in setting initial values for an object's attributes at the time of creation.

Relevant ATL Skills:

- **Thinking Skills:**
 - **Understanding concepts:** Students will develop an understanding of the fundamental concepts of classes, objects, and constructors in OOP.
 - **Applying knowledge:** Students will apply their understanding by writing Java code to define classes and instantiate objects.
 - **Problem-solving:** Students will troubleshoot any errors encountered while coding.
- **Communication Skills:**
 - **Using terminology appropriately:** Students will learn and use correct OOP terminology (class, object, instance, constructor, attribute).
 - **Explaining code:** Students may discuss their code with peers or the teacher, requiring them to explain their logic.
- **Self-Management Skills:**

- **Organisation:** Students will organise their code and manage their time effectively during coding activities.

Learning Objectives Python

By the end of this lesson, students will be able to:

- Define the syntax for declaring a class in Python.
- Identify and declare instance attributes within a Python class.
- Define the `__init__` method (constructor) within a Python class and explain its purpose in initialising object state.
- Write code to instantiate objects (create instances) of a defined Python class by calling the class name.
- Explain the role of the `__init__` method in setting initial values for an object's attributes at the time of creation.

Relevant ATL Skills:

- **Thinking Skills:**
 - **Understanding concepts:** Students will develop an understanding of the fundamental concepts of classes, objects, and the `__init__` method in OOP.
 - **Applying knowledge:** Students will apply their understanding by writing Python code to define classes and instantiate objects.
 - **Problem-solving:** Students will troubleshoot any errors encountered while coding.
- **Communication Skills:**
 - **Using terminology appropriately:** Students will learn and use correct OOP terminology (class, object, instance, `__init__` method, attribute).
 - **Explaining code:** Students may discuss their code with peers or the teacher, requiring them to explain their logic.
- **Self-Management Skills:**
 - **Organisation:** Students will organise their code and manage their time effectively during coding activities.

Lesson Resources

Presentation Java

[Link to presentation](#)

Presentation Python

[Link to presentation](#)

eBook

B 3.1.4 Classes and instantiation Java

B 3.1.4 Classes and instantiation Python

Other Resources

[B 3.1.4 Worksheet Java](#)

[B 3.1.4 Worksheet Java Answers](#)

[B 3.1.4 Worksheet Python](#)

[B 3.1.4 Worksheet Python Answers](#)

Lesson Planning

Lesson Plan Table (Java):

Time	Activity	Content	Covered Slide(s)
0-5 mins	Introduction & Objectives	Teacher introduces the topic of defining classes and instantiating objects in Java, outlines the learning objectives and relevant ATL skills for the lesson.	1
5-15 mins	Explaining Classes & Objects	Teacher uses slides to explain the concepts of classes and objects as the foundation of OOP in Java, using analogies.	2
15-25 mins	Defining a Java Class	Teacher explains the syntax for declaring a Java class and the role of attributes (instance variables), including basic data types and access modifiers.	3, 4
25-35 mins	Constructors in Java	Teacher explains the purpose and syntax of constructors in Java, including the default constructor and how to define parameterized constructors.	5, 6, 7
35-45 mins	Instantiating Objects	Teacher explains how to instantiate objects of a class in Java using the new keyword and the constructor, providing examples.	8, 9
45-55 mins	Worksheet Activity (Java)	Students work individually or in pairs on the provided Java worksheet, which includes tasks on defining simple classes and instantiating objects with appropriate constructors.	N/A
55-60 mins	Wrap-up & Q&A	Teacher reviews the key concepts, answers student questions, and briefly introduces the next steps (e.g.,	10

Time	Activity	Content	Covered Slide(s)
		accessing object attributes, calling methods). Formative assessment through quick questions.	

Lesson Plan Table (Python):

Time	Activity	Content	Covered Slide(s)
0-5 mins	Introduction & Objectives	Teacher introduces the topic of defining classes and instantiating objects in Python, outlines the learning objectives and relevant ATL skills for the lesson.	1
5-15 mins	Explaining Classes & Objects	Teacher uses slides to explain the concepts of classes and objects as the foundation of OOP in Python, using analogies.	2
15-25 mins	Defining a Python Class	Teacher explains the syntax for declaring a Python class.	3
25-35 mins	The <code>__init__</code> Method	Teacher explains the purpose and syntax of the <code>__init__</code> method (constructor) in Python and how to define instance attributes within it, including the role of <code>self</code> .	4, 5, 6
35-45 mins	Instantiating Objects	Teacher explains how to instantiate objects of a class in Python by calling the class name with arguments for the <code>__init__</code> method, providing examples.	7, 8
45-55 mins	Worksheet Activity (Python)	Students work individually or in pairs on the provided Python worksheet, which includes tasks on defining simple classes and instantiating objects with appropriate <code>__init__</code> methods.	N/A
55-60 mins	Wrap-up & Q&A	Teacher reviews the key concepts, answers student questions, and briefly introduces the next steps (e.g., accessing object attributes, calling methods). Formative assessment through quick questions.	9

Notes for Teacher: Overcoming Potential Challenges

- **Abstraction of Concepts:** Some students may struggle with the abstract nature of classes and objects. Using real-world analogies consistently (like blueprints and houses/cakes) can help make the concepts more tangible.
- **Syntax Errors:** Expect students to make syntax errors, especially with the specific requirements of Java (e.g., semicolons, access modifiers, `new` keyword) or Python (e.g., indentation, the `self` parameter, the double underscores in `__init__`). Encourage careful typing and the use of IDEs with syntax highlighting.
- **Understanding Constructors/`__init__`:** Students might not fully grasp the purpose of the constructor/`__init__` method. Emphasise that it's the first piece of code that runs when an object is created and its primary job is to set up the initial state of the object. Show examples of what happens if you don't have a constructor (default values in Java, or no initialisation in Python without `__init__`).
- **Distinguishing Class and Object:** Reinforce the difference between a class (the template) and an object (a specific instance of that template). Use diagrams or visual aids if necessary.
- **`self` in Python:** The `self` parameter in Python can be a point of confusion for beginners. Explain that it's a reference to the instance of the object being created and is necessary to access and modify the object's attributes.
- **Case Sensitivity:** Remind students that both Java and Python are case-sensitive languages, so `ClassName` is different from `classname` and `attributeName` is different from `attributename`.
- **Differentiation:** For students with less programming experience, provide more scaffolding during the worksheet activity, perhaps with partially completed code or more step-by-step instructions. For more advanced students, encourage them to think about additional attributes or more complex initialisation logic in their chosen object for Activity 2.
- **Error Messages:** Encourage students to read and try to understand error messages they encounter. Guide them on how to interpret common error types related to class definition and object instantiation.
- **Time Management:** Remind students to manage their time effectively during the worksheet activity to ensure they have a chance to attempt all tasks.
- **Formative Assessment:** Circulate during the worksheet activity to observe students' understanding and provide individual feedback. Use quick verbal questions at the end of the lesson to check for understanding of the key concepts.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.
